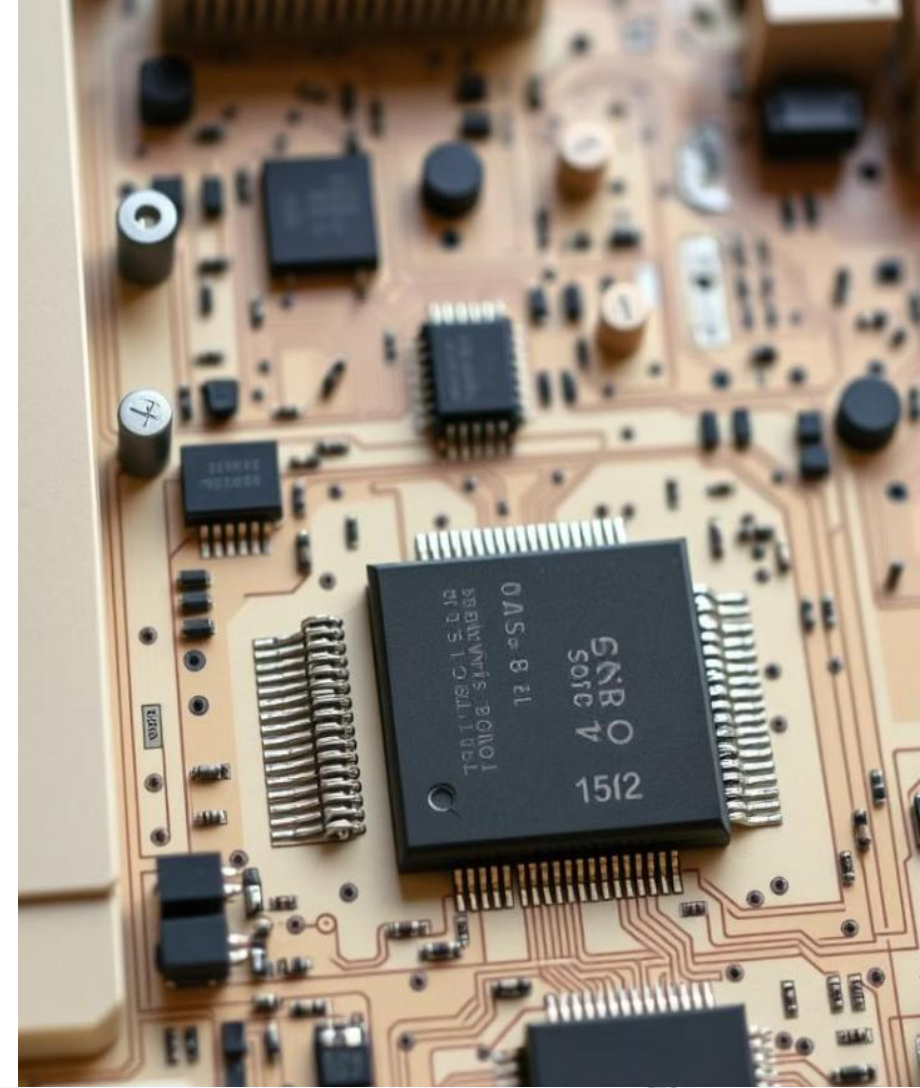


Course Title: Parallel and Cloud Computing

Course Number: CABSX05003

Parallel Programming with OpenMP



Dr. Asif Irshad Khan

Associate Professor
Department of Computer Science



airshad.cs@amu.ac.in



www.amu.ac.in



What is OpenMP?

OpenMP is a standard API (Application Programming Interface) used to write **shared-memory parallel** programs in C, C++, and Fortran.

It Consists of



Compiler Directives

Special instructions (pragmas) added in the code that tell the compiler how to run code in parallel.



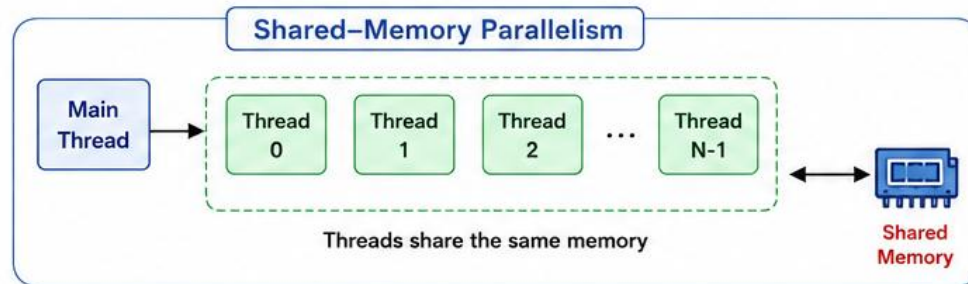
Runtime Routines

Library functions that provide information about threads, timers, and environment.



Environment Variables

External settings that control the behavior of the OpenMP runtime system.



Key Points

- ✓ OpenMP is a de-facto (widely adopted) standard for shared-memory programming.
- ✓ It works on multicore CPUs and other shared-memory architectures.
- ✓ Simple to use: add small directives to existing sequential code.
- ✓ Improves performance by executing tasks in parallel.

OpenMP Core Syntax

1. Compiler Directives

Most constructs in OpenMP are compiler directives.

General form:

```
#pragma omp construct [clause [clause] ...]
```

Example:

```
#pragma omp parallel num_threads(4)
```

2. Header File

Function prototypes and types are declared in:

```
#include <omp.h>
```

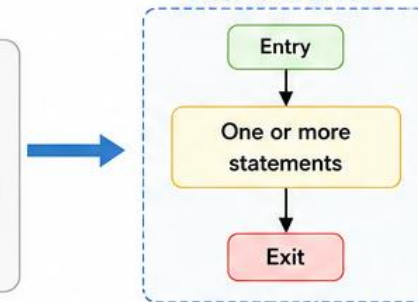
3. Structured Block

Most OpenMP constructs apply to a “structured block”.

Example

```
#pragma omp parallel
{
    // statements
    if (condition)
        exit(0); // allowed
    // more statements
}
```

Structured Block



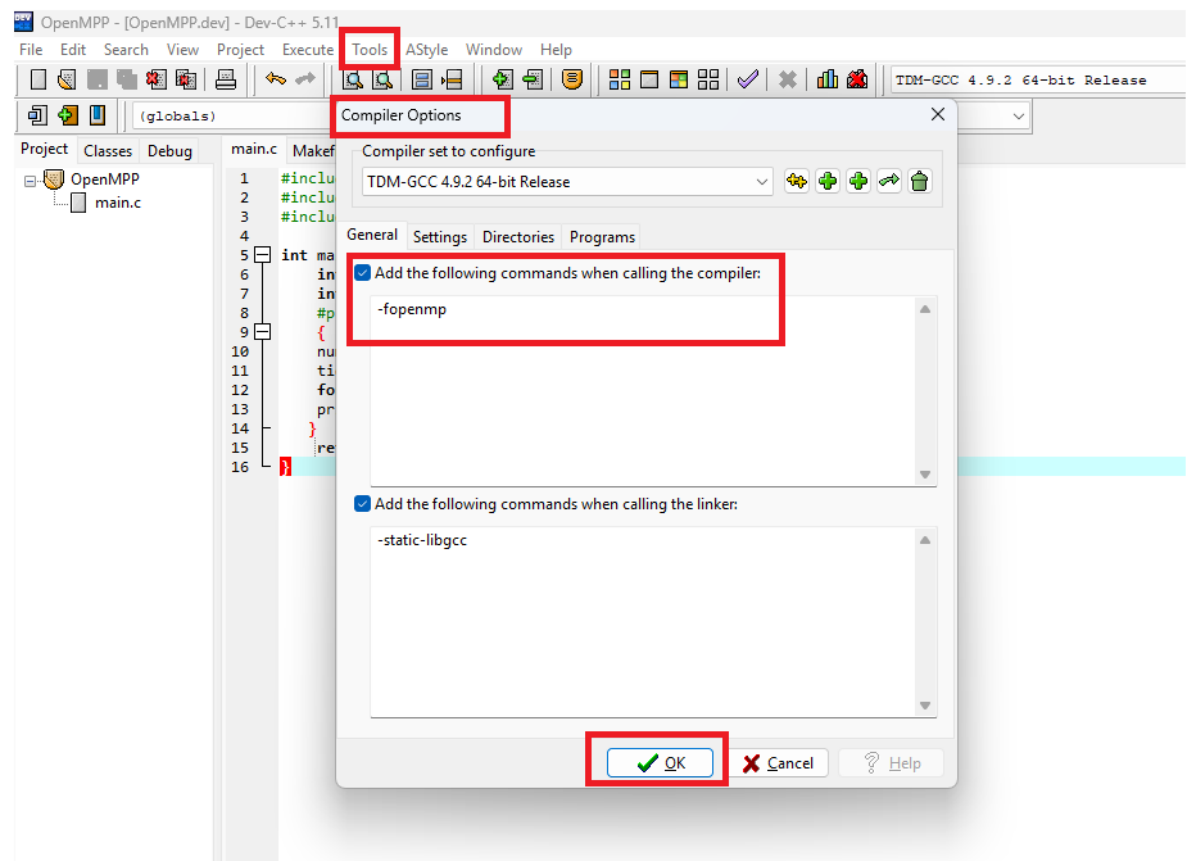
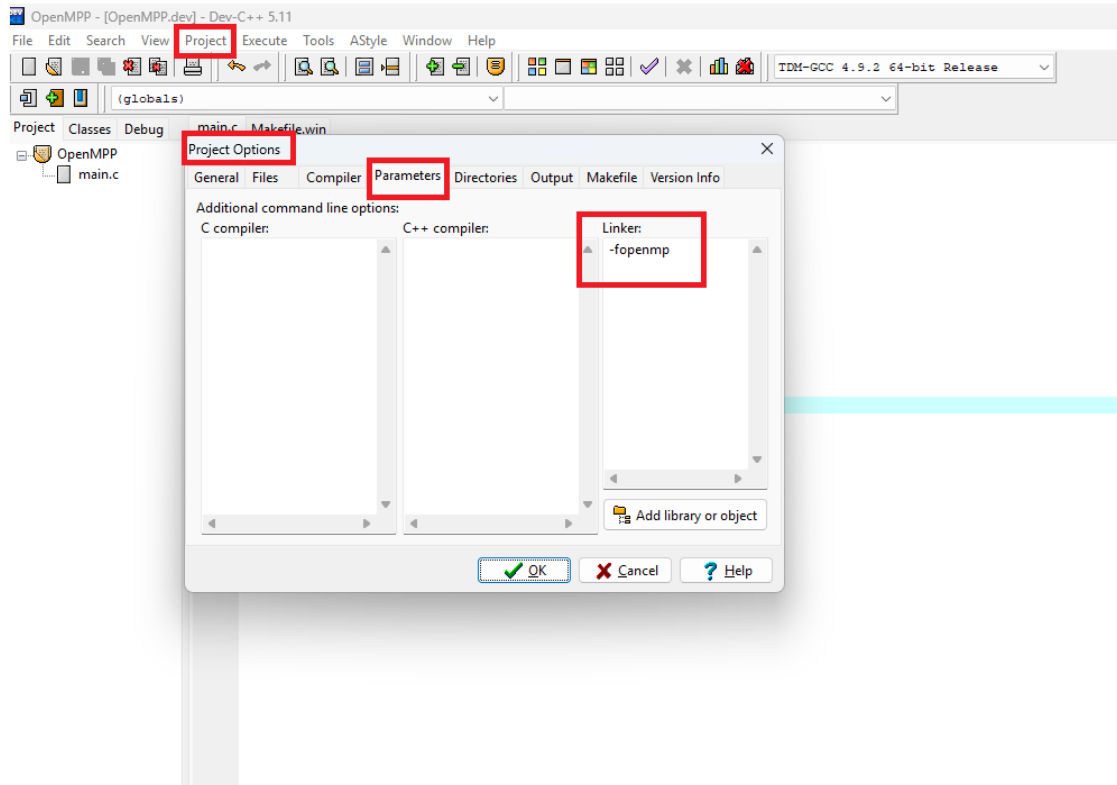
- ✓ One entry point at the top.
- ✓ One exit point at the bottom.
- ✓ It is OK to have exit() inside the block.

Key Points

- ✓ Use `#pragma omp ...` directives to define parallel regions and work.
- ✓ Include `<omp.h>` to access OpenMP library functions.
- ✓ OpenMP constructs work on structured blocks with single entry and exit.
- ✓ `exit()` is allowed inside the structured block.

OpenMP – Dev C++ environment setting

To enable and configure **OpenMP in Dev-C++**, you must explicitly pass the **-fopenmp** flag to both the **compiler** and the **linker**.



Exercise 1, Part A: Hello world

✓ Goal: Verify that your environment works.

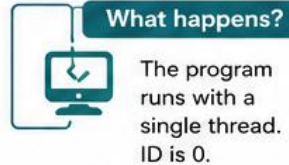
 Task: Write a program that prints "hello world".

hello.c

```
#include <stdio.h>

int main()
{
    int ID = 0;
    printf("hello(%d) ", ID);
    printf("world(%d) \n", ID);

    return 0;
}
```



Sample Output

hello(0) world(0)

Key Points

- ✓ This confirms your compiler and environment are set up correctly.

Exercise 1, Part B: Hello world (OpenMP)

✓ Goal: Verify that your OpenMP environment works.

 Task: Write a multithreaded program that prints "hello world".

hello_omp.c

```
#include <stdio.h>
#include <omp.h>

int main()
{
    #pragma omp parallel
    {
        int ID = 0;
        printf("hello(%d) ", ID);
        printf("world(%d) \n", ID);
    }

    return 0;
}
```

 Compilation (examples)

```
gcc -fopenmp hello_omp.c -o hello_omp
gcc -mp hello_omp.c -o hello_omp
pgcc -mp hello_omp.c -o hello_omp
icx /Qopenmp hello_omp.c -o hello_omp
```

 What happens?

The program runs with multiple threads (default number depends on your system).

Sample Output (order may vary)

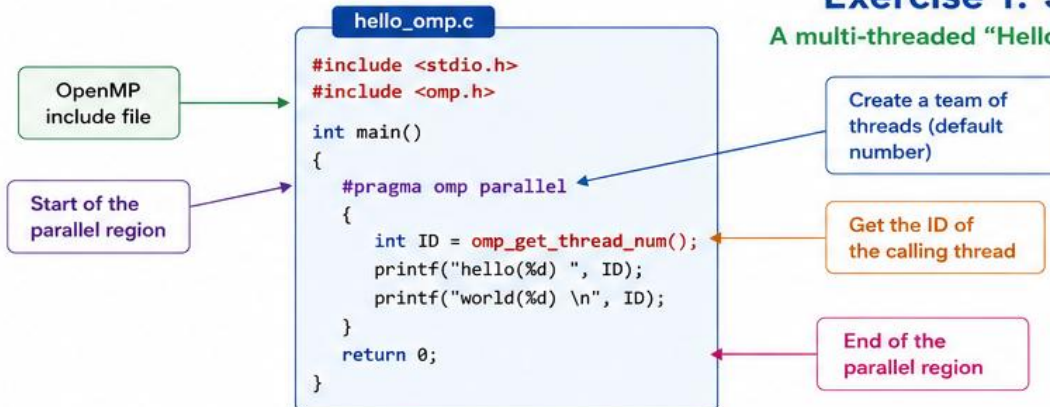
```
hello(0) world(0) hello(0) world(0)
hello(0) world(0) hello(0) world(0)
...
```

Key Points

- ✓ #include <omp.h> is required to use OpenMP.
- ✓ #pragma omp parallel creates a parallel region.
- ✓ Compilation requires OpenMP flags (examples shown above).

Exercise 1: Solution

A multi-threaded "Hello world" program



How it works

- 1 #pragma omp parallel creates a parallel region.
- 2 All threads in the team execute the block inside { }.
- 3 omp_get_thread_num() returns a unique ID to each thread.
- 4 Each thread prints "hello world" with its own ID.

Sample Output (order may vary)

```
hello(0) world(0) hello(2) world(2) hello(3) world(3)
hello(1) world(1) hello(4) world(4) ...
```

Key Takeaways



OpenMP uses simple directives to enable parallelism.



Parallel regions run with multiple threads concurrently.



omp_get_thread_num() identifies each thread.



Easy to start: add a few lines and compile with OpenMP flags!

What Happens Internally in OpenMP?

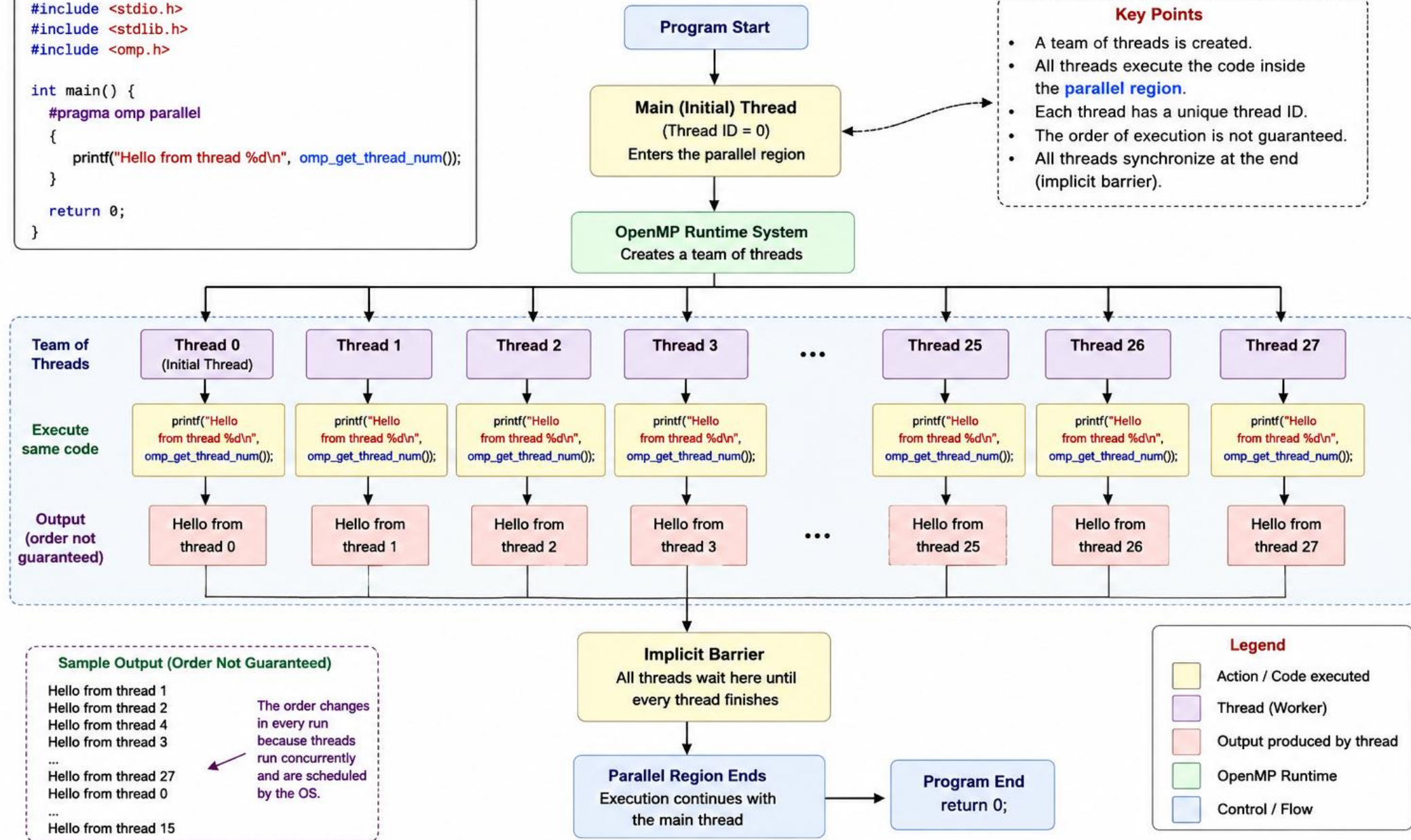
Execution of a Parallel Region

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Hello from thread %d\n", omp_get_thread_num());
    }
    return 0;
}
```

Key Points

- A team of threads is created.
- All threads execute the code inside the **parallel region**.
- Each thread has a unique thread ID.
- The order of execution is not guaranteed.
- All threads synchronize at the end (implicit barrier).



Hello World! in Parallel (OpenMP)

C Program

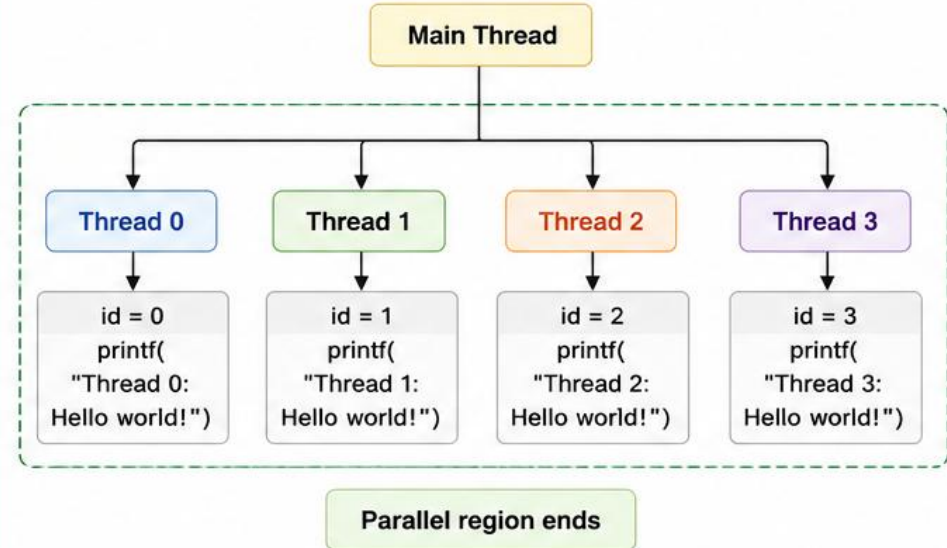
```
1 #include <stdio.h>
2 #include <omp.h>
3
4 void main()
5 {
6     omp_set_num_threads(4); // Set number of threads to 4
7     int id;
8     #pragma omp parallel
9     {
10         id = omp_get_thread_num(); // Get thread ID
11         printf("Thread %d: Hello world!\n", id);
12     }
13     return;
14 }
```

Total number of threads = 4

Each thread gets a unique thread ID (0 to 3)

What Happens?

OpenMP creates a team of 4 threads. Each thread executes the same block of code inside the parallel region.



Compile and Run



Compile (using GCC):

```
gcc -fopenmp hello_world.c -o hello_world
```

Run the program:

```
./hello_world
```

Sample Output (Order may vary)

```
Thread 0: Hello world!
Thread 1: Hello world!
Thread 2: Hello world!
Thread 3: Hello world!
```

OR

```
Thread 2: Hello world!
Thread 0: Hello world!
Thread 3: Hello world!
Thread 1: Hello world!
```

Note: The order of output may change from run to run.

Key Takeaways



`omp_set_num_threads(n)` sets the number of threads to be used.



`#pragma omp parallel` creates a team of threads that share the workload.



`omp_get_thread_num()` returns the unique ID of the calling thread.



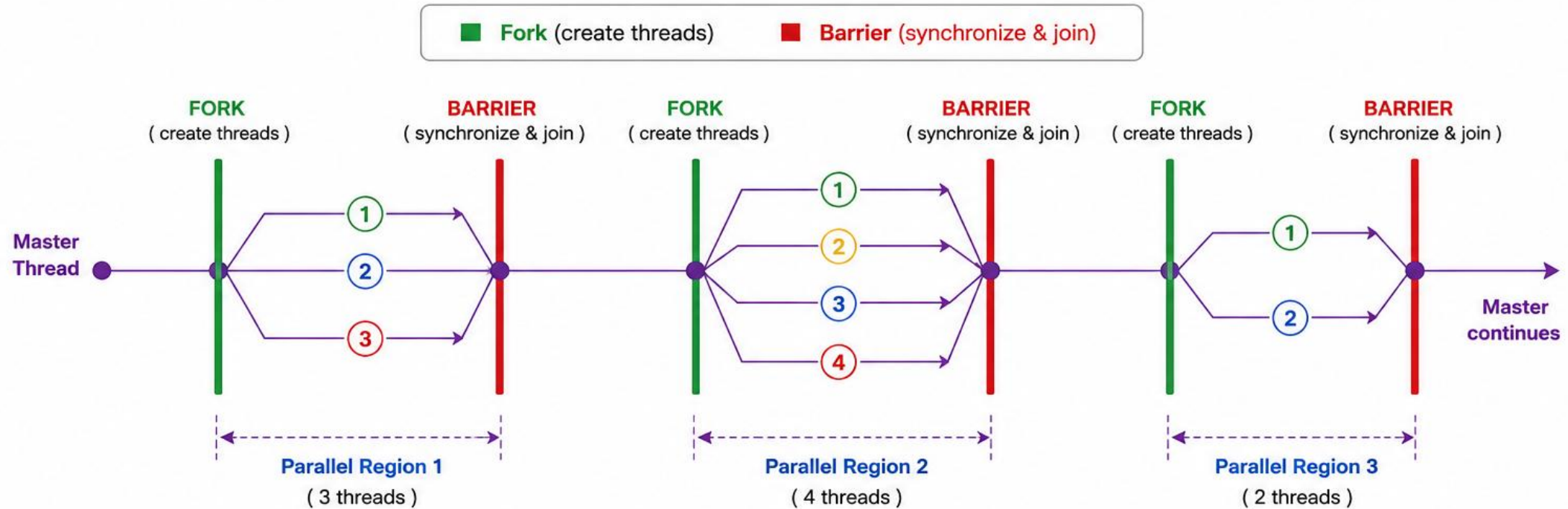
All threads execute the same code inside the parallel region.



Simple "Hello World" is the first step toward parallel programming!

Fork/Join Model in OpenMP

OpenMP programs start with a single **master thread**.
At each parallel region, the master thread **forks** a team of threads.
At the end of the region, all threads synchronize (**barrier**) and **join** back to the master thread.



Key Takeaways



FORK: At the start of a parallel region, the master creates a team of worker threads.



PARALLEL EXECUTION: All threads execute the statements in the parallel region concurrently.



BARRIER: At the end of the parallel region, all threads synchronize. Fast threads wait for the slowest one.



JOIN: After synchronization, all threads terminate (except the master), and execution continues with the master thread.

OpenMP Fork/Join Model

OpenMP follows the **fork/join model** for parallel execution.

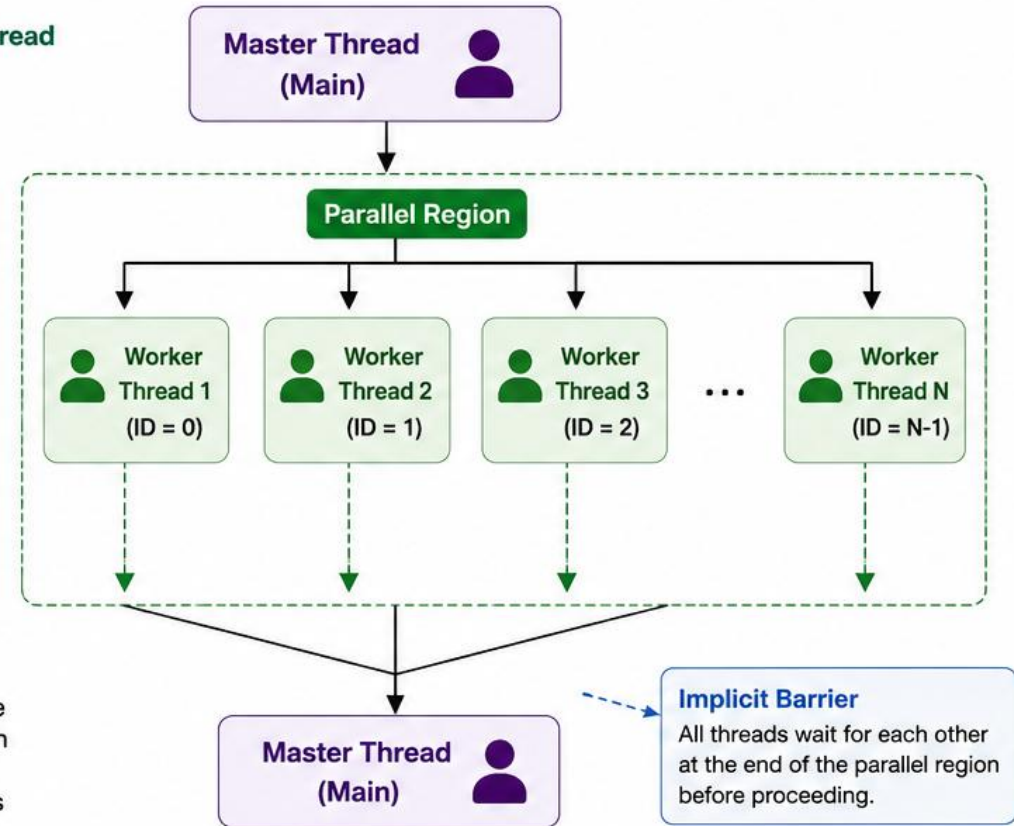
Key Points

- 1 Start with a single thread**
OpenMP programs start with a single **master thread**.
- 2 Fork: Create worker threads**
At the start of a parallel region, the master thread creates a team of parallel “**worker**” threads. This is called **FORK**.
- 3 Execute in parallel**
All threads (master + workers) execute the statements in the parallel region **concurrently**.
- 4 Join: Synchronize and merge**
At the end of the parallel region, all threads synchronize and terminate, and control returns to the master thread. This is called **JOIN**.

 **Implicit barrier:** There is an implicit barrier at the end of a parallel region.

Visual Overview

- 1 Single Master Thread**
Program starts.
- 2 FORK**
Master creates a team of worker threads at the start of the parallel region.
- 3 Execute in Parallel**
All threads execute the statements in the parallel region **concurrently**.
- 4 JOIN**
All threads synchronize at the end of the region and **join the master** thread. Control returns to the **master**.



In Short



Start with one **master** thread.



Fork: Master creates multiple **worker** threads.



All threads execute the work in **parallel** inside the region.



Join: Threads synchronize and merge back.

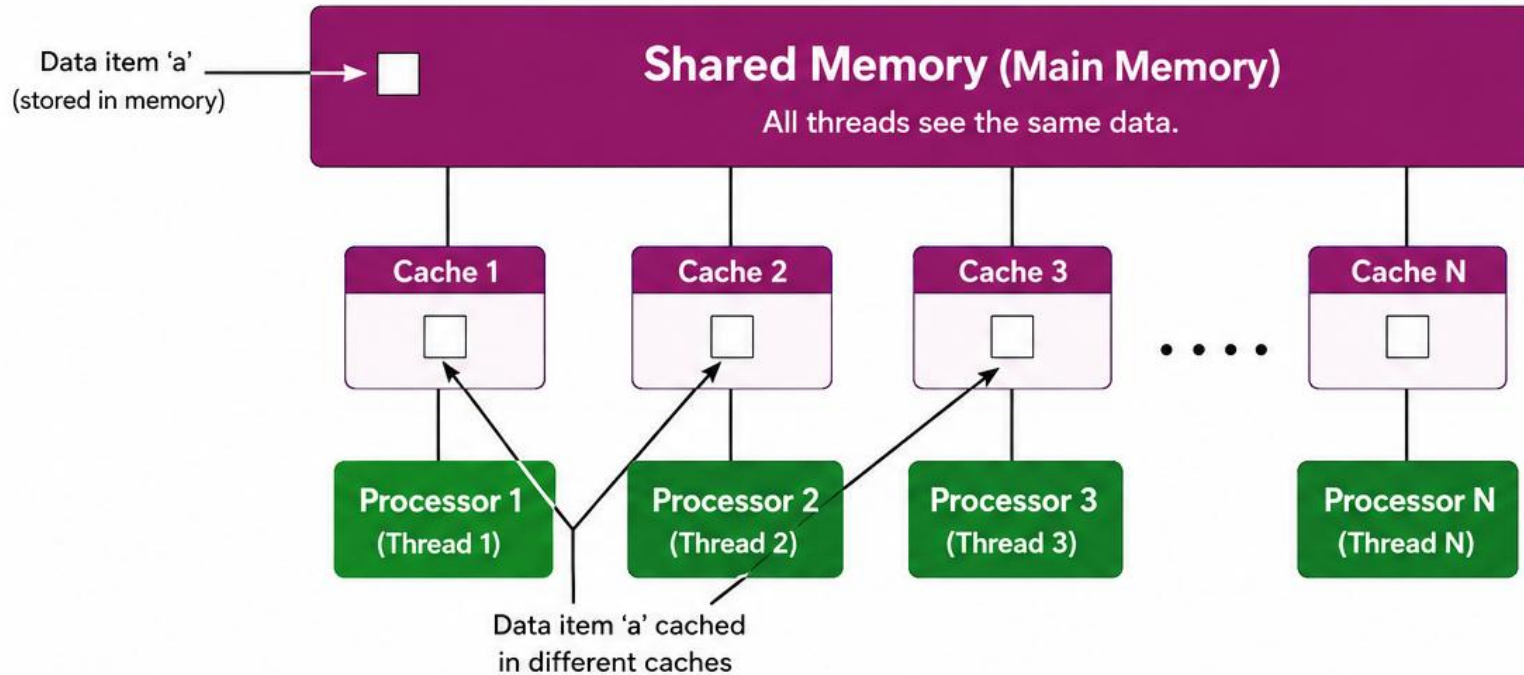
The **Fork/Join Model** is the basic execution model used by OpenMP to manage parallelism.

FORK vs JOIN

Stage	Meaning
Start	One master thread
FORK	Master creates a team of worker threads
Parallel Region	Threads execute work concurrently
Implicit Barrier	Threads wait for one another
JOIN	Worker threads finish and execution continues

OpenMP Memory Model

- OpenMP uses a **shared memory model**.
- All threads **share the same address space**, but due to hardware, it can get complicated.



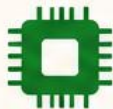
How it works

- 1 A variable (e.g., 'a') is stored in shared memory.
- 2 Each processor (core) may keep its own copy of the data in its local cache.
- 3 All threads access the same memory location, but may read/write through their own cache.
- 4 Caches help performance but can cause complexity (e.g., stale data, consistency).

Key Takeaways



OpenMP is a **shared memory** parallel programming model.



All threads share the **same address space** and global memory.



Multiple copies of data may exist in main memory, caches, or registers.



Caches improve performance but may introduce complexity.



Understanding the memory hierarchy is important for **correctness and performance** (synchronization, consistency).



In short: Threads share the same memory, but hardware (caches, registers) keeps multiple copies of data, making memory behavior more complex.

OpenMP Threads vs Cores

What are threads, cores, and how do they relate?

Key Concepts



Thread

An independent sequence of execution of program code.

- A block of code with one entry and one exit.



Core (CPU Core)

A physical processing unit in a CPU.

- Cores execute threads.



Abstraction

Threads are a more abstract concept than cores/CPUs.



Mapping

OpenMP threads are mapped onto physical CPU cores by the system.



Flexibility

It is possible to map more than one thread on a single core.



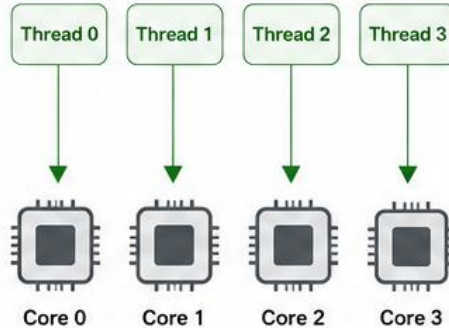
Best Practice

In practice, best performance is usually achieved with one-to-one mapping (1 thread per core).

How OpenMP Threads Map to CPU Cores

1) One-to-One Mapping (Recommended)

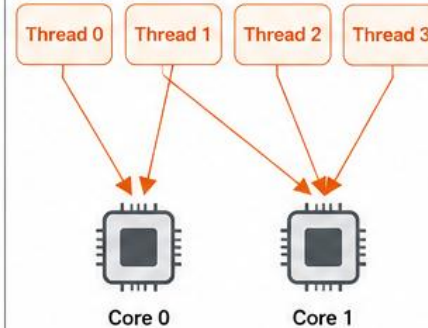
1 thread per core



Best performance and no oversubscription.

2) Many-to-One Mapping

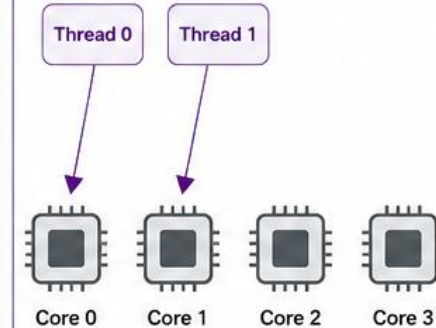
Multiple threads on one core



Oversubscription may cause context switching and lower performance.

3) Fewer Threads than Cores

Not all cores are used



Some cores are idle. Not ideal for performance.

Key Takeaways



A **thread** is a sequence of instructions.



A **core** is a physical execution unit.



Threads are abstract; cores are physical.



More than one thread can run on a core, but it may reduce efficiency.



Aim for one-to-one mapping for best performance.

More About OpenMP Threads

Setting the Number of OpenMP Threads



1) Using Environment Variable

Set the environment variable **OMP_NUM_THREADS** before running the program.

```
$ export OMP_NUM_THREADS=4 # Linux / macOS
$ ./a.out
```

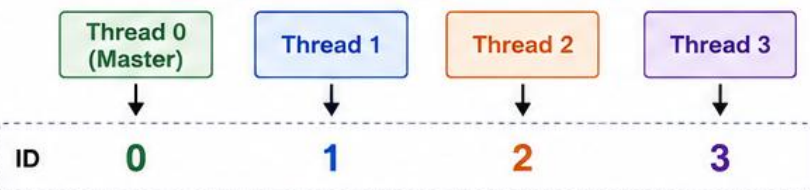


2) Using Runtime Function

Set the number of threads in your program before entering the parallel region.

```
#include <omp.h>
...
omp_set_num_threads(n); // set number of threads
```

Example: 4 Threads in a Parallel Region



i Thread IDs are unique within a team and help threads identify their work.

Useful OpenMP Runtime Functions



omp_get_num_threads()

- Returns the number of threads in the current parallel region.
- If called outside a parallel region, returns 1.

```
int nt = omp_get_num_threads();
printf("Number of threads = %d\n", nt);
```



omp_get_thread_num()

- Returns the ID of the calling thread in the team.
- Value is between [0, n-1], where n = number of threads.
- The master thread always has ID 0.

```
int tid = omp_get_thread_num();
printf("Thread ID = %d\n", tid);
```

Summary

Function / Variable	Purpose
OMP_NUM_THREADS	Environment variable to set number of threads.
omp_set_num_threads(n)	Runtime function to set number of threads.
omp_get_num_threads()	Get number of threads in current parallel region (or 1 outside).
omp_get_thread_num()	Get ID of calling thread in the team [0, n-1]; master has ID 0.



Key Takeaways

You can control the number of OpenMP threads using an environment variable or a runtime function.

omp_get_num_threads() tells you how many threads are in the current team.

omp_get_thread_num() gives each thread a unique ID from 0 to n-1 (master is 0).

These functions are essential for writing efficient and flexible parallel programs.

Example#

/* Your program is a simple example of using OpenMP to create a parallel region in a C program. In this code, each thread will print a message indicating its thread number. */

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
//Include the OpenMP Header:
```

```
#include <omp.h>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    // Parallel region
```

```
    #pragma omp parallel
```

```
    {
```

```
        printf("Hello from thread %d\n", omp_get_thread_num());
```

```
    }
```

```
    return 0;
```

```
}
```

OpenMP Example: Print Thread Numbers

Example Code

```
1  /* Your program is a simple example of using OpenMP to create
2     a parallel region in a C program. In this code, each thread will
3     print a message indicating its thread number. */
4  #include <stdio.h>
5  #include <stdlib.h>
6  //Include the OpenMP Header:
7  #include <omp.h>
8
9  int main(int argc, char *argv[])
10 {
11     // Parallel region
12     #pragma omp parallel
13     {
14         printf("Hello from thread %d\n", omp_get_thread_num());
15     }
16     return 0;
17 }
```

Required header files for standard functions and OpenMP.

Program starts execution from here.

Starts a parallel region: all threads execute the block inside.

Returns the unique thread ID of the calling thread.

Breakdown of the Code

1



Headers

The program includes necessary headers: **stdio.h** for standard input/output functions, **stdlib.h** for general utilities, and **omp.h** for OpenMP-specific functions.

2



main Function

The program's entry point is the main function, which also accepts command-line arguments.

3



Parallel Region

The **#pragma omp parallel** directive creates a parallel region. The code within this block will be executed by multiple threads concurrently.

4



Thread Identification

omp_get_thread_num() returns the unique thread number (ID) of each thread. In the printf statement, each thread prints a message indicating its thread number.

What it does?



Creates a team of threads. Each thread prints a message showing its own thread ID.

Sample Output (Order may vary)

```
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3
...
```

(Up to N threads depending on your system)

Order is not guaranteed because threads run concurrently.

★ Key Takeaways

- ✓ **#pragma omp parallel** creates a team of threads.
- ✓ Each thread executes the code inside the parallel region.
- ✓ **omp_get_thread_num()** helps identify each thread.
- ✓ Number of threads = set by environment or **omp_set_num_threads()**.
- ✓ Output order is not deterministic (may change every run).

Example#

/*Your C program utilizes OpenMP to perform parallel addition of elements from two arrays (a and b) and stores the results in a third array (c). Each thread is responsible for adding corresponding elements from the a and b arrays and printing the result.*/

```
#include<stdio.h>
```

```
#include<omp.h>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    int a[5]={1,2,3,4,5};
```

```
    int b[5]={6,7,8,9,10};
```

```
    int c[5];
```

```
    int tid;
```

```
    #pragma omp parallel num_threads(5)
```

```
    {
```

```
        tid=omp_get_thread_num(); /* get the thread id*/
```

```
        c[tid]=a[tid]+b[tid]; /*each operation on individual core*/
```

```
        printf("c[%d]=%d\n",tid,c[tid]);
```

```
    }
```

```
}
```

OpenMP Example: Parallel Addition of Two Arrays

This C program uses OpenMP to perform parallel addition of elements from two arrays (a and b) and stores the results in a third array (c). Each thread is responsible for adding corresponding elements from the arrays and printing the result.

C Program

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main(int argc, char *argv[])
5  {
6      int a[5] = {1, 2, 3, 4, 5};
7      int b[5] = {6, 7, 8, 9, 10};
8      int c[5];
9      int tid;
10
11     #pragma omp parallel num_threads(5)
12     {
13         tid = omp_get_thread_num(); // get the thread id
14         c[tid] = a[tid] + b[tid];    // add on individual core
15         printf("c[%d] = %d\n", tid, c[tid]);
16     }
17 }
18 }
```

Arrays and variables are declared.

Creates a parallel region with 5 threads.

Each thread works on its own index (tid).

Each thread prints its part of the result.



Breakdown of the Code

1



Array Initialization

Two arrays, a and b, are initialized with values:
 $a = \{1, 2, 3, 4, 5\}$ $b = \{6, 7, 8, 9, 10\}$
An array c is declared to store the result of adding elements from a and b.

2



Parallel Region

The `#pragma omp parallel num_threads(5)` directive creates a parallel region with 5 threads.

3



Thread ID

`tid = omp_get_thread_num()` gets the unique thread ID for each thread (ranging from 0 to 4).

4



Array Addition

Each thread performs addition on elements of a and b arrays using its unique thread ID as an index.

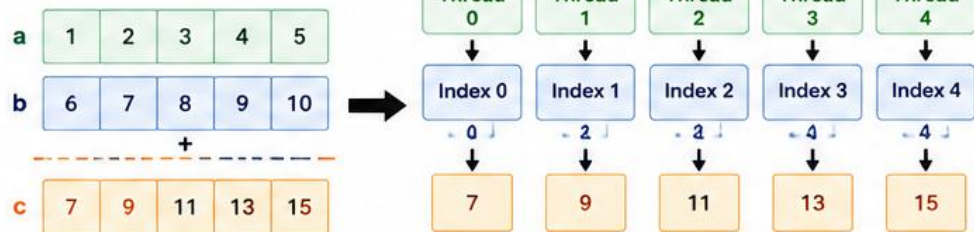
5



Output

Each thread prints its part of the result.

How it Works



Sample Output (Order may vary)

```
c[0] = 7
c[1] = 9
c[2] = 11
c[3] = 13
c[4] = 15
```

(Order of lines may change in every run)

Key Takeaways

- ✓ OpenMP makes it easy to run code in parallel.
- ✓ `#pragma omp parallel` creates a team of threads.
- ✓ `omp_get_thread_num()` gives each thread a unique ID.
- ✓ Each thread works independently on its own data.
- ✓ Results may be printed in any order.
- ✓ Useful for data-parallel tasks like array operations.

Example#

/*Write a program that multiplies each element of an array by 2 using multiple threads.*/

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main(int argc, char *argv[]) {  
    int array[5] = {1, 2, 3, 4, 5}; // Array of numbers  
    int i; // Loop counter
```

```
    // Parallel region with simple for loop
```

```
    #pragma omp parallel for
```

```
    for (i = 0; i < 5; i++) {  
        array[i] *= 2; // Multiply each element by 2  
        printf("Thread %d: array[%d] = %d\n", omp_get_thread_num(), i, array[i]);  
    }
```

```
    // Print the final array after modification
```

```
    printf("Final array values:\n");
```

```
    for (i = 0; i < 5; i++) {  
        printf("%d ", array[i]);
```

```
    }  
    printf("\n");
```

```
    return 0;
```

```
}
```

Example



Write a program that multiplies each element of an array by 2 using multiple threads.

C Program

```
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    int array[5] = {1, 2, 3, 4, 5}; // Array of numbers
    int i; // Loop counter

    // Parallel region with simple for loop
    #pragma omp parallel for
    for (i = 0; i < 5; i++) {
        array[i] *= 2; // Multiply each element by 2
        printf("Thread %d: array[%d] = %d\n",
               omp_get_thread_num(), i, array[i]);
    }

    // Print the final array after modification
    printf("\nFinal array values:\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", array[i]);
    }
    printf("\n");

    return 0;
}
```

Sample Output (with 4 threads)

```
Thread 0: array[0] = 2
Thread 2: array[2] = 6
Thread 1: array[1] = 4
Thread 3: array[3] = 8
Thread 0: array[4] = 10
```

Final array values:

```
2 4 6 8 10
```



Note:

The order of thread prints may vary because threads execute concurrently.

Here's a breakdown of the code:

1 Array Initialization

The array is initialized with the values {1, 2, 3, 4, 5}.

2 Parallel Region

The `#pragma omp parallel for` directive is used to indicate that the following `for` loop should be executed in parallel. Each iteration of the loop is handled by a different thread.

3 Multiplication Operation

Each thread multiplies its assigned element of the array by 2.

4 Printing Thread Information

Inside the loop, each thread prints its ID (using `omp_get_thread_num()`) and the modified value of the array element it processed.

5 Final Array Output

After the parallel region, the main thread prints the final values of the array.

Key Takeaways

- ✓ `#pragma omp parallel for` parallelizes a loop, distributing iterations among threads.
- ✓ `omp_get_thread_num()` gives the ID of the calling thread (from 0 to n-1).
- ✓ Threads execute concurrently, so the print order may vary.
- ✓ After the parallel region, all threads synchronize, and execution continues with the main thread.

Example#

```
/*Write a program Calculating the Cube of Array Elements Using OpenMP.*/
```

```
#include <stdio.h>
```

```
#include <omp.h> // Include the OpenMP header
```

```
int main() {
```

```
    int n = 6;
```

```
    int a[] = {1, 2, 3, 4, 5, 6}; // Input array
```

```
    int c[n]; // Array to store the cubes
```

```
    // Parallel region to compute cubes
```

```
    #pragma omp parallel for
```

```
    for (int i = 0; i < n; i++) {
```

```
        c[i] = a[i] * a[i] * a[i];
```

```
    }
```

```
    // Print results
```

```
    printf("Cubed elements:\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", c[i]);
```

```
    }
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

Example



Write a program Calculating the Cube of Array Elements Using OpenMP.

C Program

```
#include <stdio.h>
#include <omp.h> // Include the OpenMP header

int main() {
    int n = 6;
    int a[] = {1, 2, 3, 4, 5, 6}; // Input array
    int c[n]; // Array to store the cubes

    // Parallel region to compute cubes
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        c[i] = a[i] * a[i] * a[i]; // Calculate cube
    }

    // Print results
    printf("Cubed elements:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", c[i]);
    }
    printf("\n");

    return 0;
}
```

Sample Output

Cubed elements:
1 8 27 64 125 216

Here's a breakdown of the code:

- 1 Include OpenMP Header**
`#include <omp.h>` is included to use OpenMP functions.
- 2 Array Initialization**
The `a[]` array is initialized with the values {1, 2, 3, 4, 5, 6}.
The array `c[]` is declared to store the cube of each element.
- 3 Parallel Region**
The `#pragma omp parallel for` directive is used to parallelize the for loop. This directive splits the loop iterations among the available threads.
- 4 Calculating Cubes**
Each thread calculates the cube of the element assigned to it and stores it in the corresponding position in the `c[]` array.
- 5 Printing Results**
After the parallel computation, the results are printed to show the cubes of the original array elements.

Key Takeaways

- `#pragma omp parallel for` distributes loop iterations among threads.
- Each thread works on a portion of the data independently.
- No explicit synchronization is needed here because each thread writes to a different index of `c[]`.
- This leads to faster execution on multi-core processors.

// Hello program with Failing

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    int numt, tid;
    #pragma omp parallel
    {
        numt= omp_get_num_threads();
        tid=omp_get_thread_num();
        printf(" hello from thread %d of %d\n",tid,numt);
    }
    return 0;
}
```

// Hello program Almost always Failing

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    int numt,tid;
    int j;
    #pragma omp parallel
    {
        numt= omp_get_num_threads();
        tid=omp_get_thread_num();
        for( j=0;j<=1000000;j++);
        printf(" hello from thread %d of %d\n",tid,numt);
    }
    return 0;
}
```

// fixing Hello program

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
int main(int argc, char *argv[]) {
    int numt,tid;
    #pragma omp parallel private ( tid ) shared( numt )
    {
        numt= omp_get_num_threads();
        tid=omp_get_thread_num();
        printf(" hello from thread %d of %d\n",tid,numt);
    }
    return 0;
}
```

// Hello program again

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <omp.h>
```

```
int main(int argc, char *argv[]) {
```

```
    int numt,tid;
```

```
    numt= omp_get_num_threads();
```

```
    #pragma omp parallel
```

```
    {
```

```
        tid=omp_get_thread_num();
```

```
        printf(" hello from thread %d of %d\n",tid,numt);
```

```
    }
```

```
    return 0;
```

```
}
```

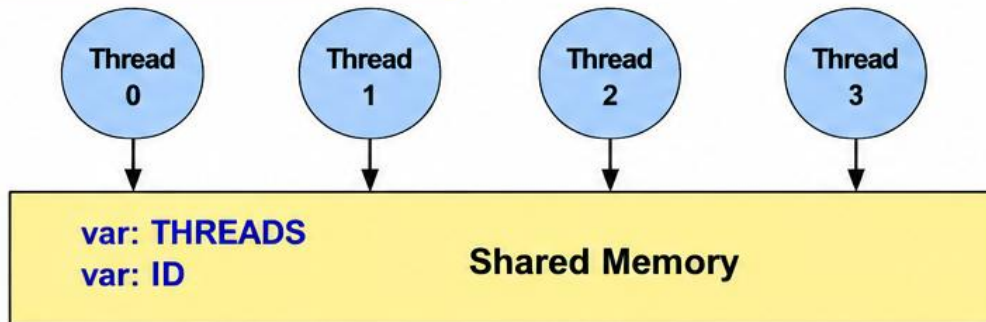
Shared Memory Model

The memory is (**logically**) shared by all the CPU's

Relevant piece of code

```
THREADS = omp_get_num_threads()  
ID = omp_get_thread_num()
```

From our example
on previous page



All threads try to access the same variable (possibly at the same time). This can lead to a race condition. Different runs of the same program might give different results because of race conditions.

Shared and Private Variables

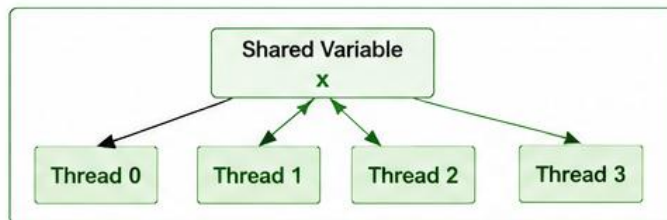
OpenMP provides a way to declare variables **private** or shared within an OpenMP block.

This is done using the following OpenMP **clauses**:



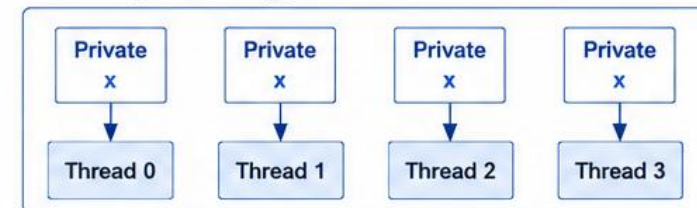
SHARED (*list*)

- All variables in *list* will be considered shared.
- **Every OpenMP thread** has access to all these variables.



PRIVATE (*list*)

- Every OpenMP thread will have its own "private" copy of variables in *list*.
- No other OpenMP thread has access to this "private" copy.



For example:

`!$OMP PARALLEL PRIVATE(a,b,c)` (Fortran)

or

`#pragma omp parallel private(a,b,c)` (C/C++)

OpenMP Constructs

OpenMP provides a set of simple but powerful constructs (**directives**) to express **parallelism** in C, C++, and Fortran programs.

1

Parallel Region



- Creates a team of threads to execute a block of code in **parallel**.
- Implicit barrier at the end (unless specified otherwise).

```
#pragma omp parallel
{
    // Code executed by all threads
}
```

2

Data Environment



- Controls how variables are shared among threads.
- Variables can be declared as **shared** or **private**.

```
#pragma omp parallel shared(a, b) private(c)
{
    // Code
}
```

3

Worksharing



- Distributes work among threads.
- Each thread performs a portion of the iterations.

```
#pragma omp for           // Distribute loop iterations
#pragma omp sections      // Distribute code sections
```

4

Synchronization



- Ensures correct coordination among threads.
- Helps prevent race conditions and maintain data consistency.

```
#pragma omp barrier      // Wait for all threads
#pragma omp critical      // Protect critical section
```



At a Glance

1

Parallel Region

Create threads and run in parallel

2

Data Environment

Define variable sharing behavior

3

Worksharing

Distribute work among threads

4

Synchronization

Coordinate threads and ensure correctness

Control Number of Threads

OpenMP provides three ways to set the number of threads.

1



Parallel Region Clause

Specify the number of threads directly in the parallel region.

```
#pragma omp parallel num_threads(integer)
```

2



Run-time Function

Set the number of threads at runtime using a function.

```
omp_set_num_threads(integer)
```

3



Environment Variable

Set the environment variable before running the program.

```
OMP_NUM_THREADS
```

Priority

Highest Priority

(Parallel region clause)

Medium Priority

(Run-time function)

Lowest Priority

(Environment variable)



Note: If more than one method is used, the one with the highest priority takes precedence.

//Sum array serial correct version

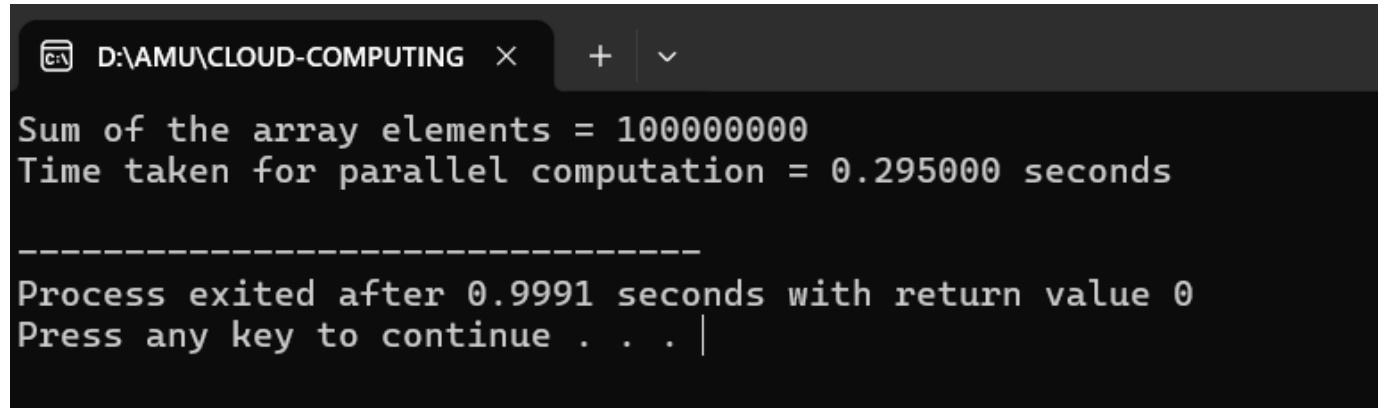
```
#include <stdio.h>
#define ARR_SIZE 100000000 // 100 million
long a[ARR_SIZE];
int main() {
    int i, tid, numt;
    long long sum = 0; // Use long long to handle large sums
    double t1, t2;

    // Initialize the array
    for (i = 0; i < ARR_SIZE; i++) {
        a[i] = 1;
    }

    t1 = omp_get_wtime(); // Start time

    for (i = 0; i < ARR_SIZE; i++) {
        sum += a[i];
    }

    t2 = omp_get_wtime(); // End time
    printf("Sum of the array elements = %lld\n", sum);
    printf("Time taken for Serial computation = %f seconds\n", t2 - t1);
    return 0;
}
```



The screenshot shows a C++ IDE window titled "D:\AMU\CLOUD-COMPUTING" with a tab for "C++". The output window displays the following text:

```
Sum of the array elements = 100000000
Time taken for parallel computation = 0.295000 seconds

-----
Process exited after 0.9991 seconds with return value 0
Press any key to continue . . . |
```

//Sum array Parallel with bugs

```
#include <stdio.h>
#include <omp.h>
#define ARR_SIZE 100000000 // Big Size // 100 million
long a[ARR_SIZE];
int main() {
    int i, tid, numt;
    long long sum = 0; // Use long long to handle large sums
    double t1, t2;
    // Initialize the array
    for (i = 0; i < ARR_SIZE; i++) {
        a[i] = 1;
    }

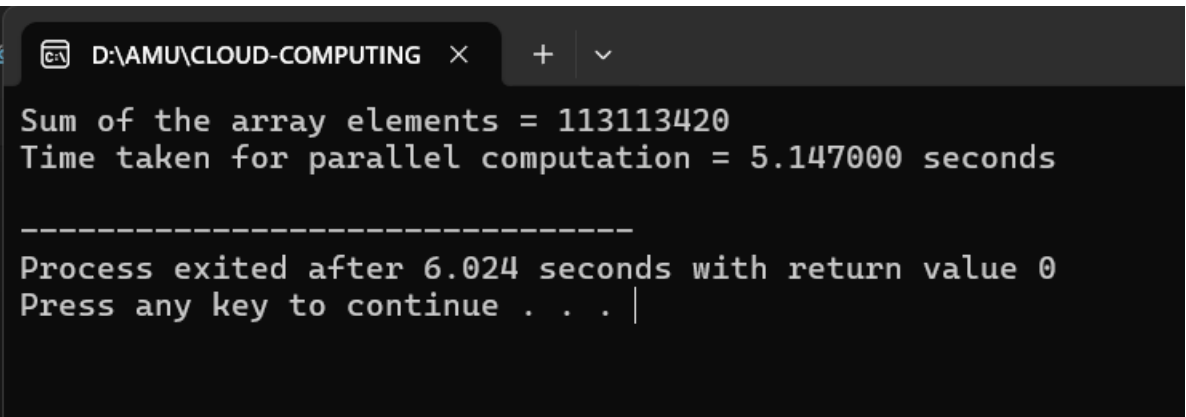
    t1 = omp_get_wtime(); // Start time
    /* Do the sum in parallel */
    #pragma omp parallel default(shared) private(i, tid)
    {
        tid = omp_get_thread_num();
        numt = omp_get_num_threads();

        for (i = 0; i < ARR_SIZE; i++) {
            sum += a[i];
        }
    }
    t2 = omp_get_wtime(); // End time
    printf("Sum of the array elements = %lld\n", sum);
    printf("Time taken for parallel computation = %f seconds\n", t2 - t1);
    return 0;
}
```

What are fundamental issues in this code ?

Race condition

Cache coherency



```
D:\AMU\CLOUD-COMPUTING x + v
Sum of the array elements = 113113420
Time taken for parallel computation = 5.147000 seconds

-----
Process exited after 6.024 seconds with return value 0
Press any key to continue . . . |
```

//Sum array Parallel without bugs

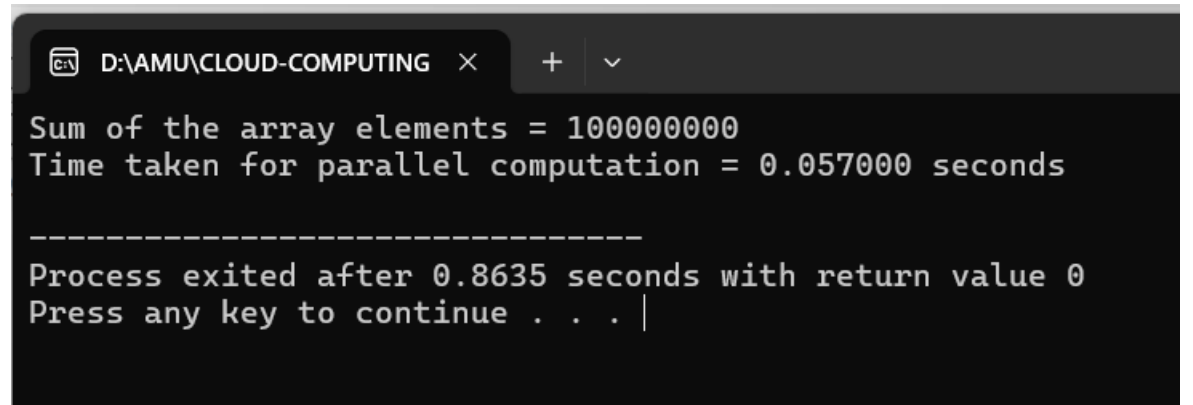
```
#include <stdio.h>
#include <omp.h>

#define ARR_SIZE 100000000 // 100 million
long a[ARR_SIZE];
int main() {
    int i, tid, numt;
    long long sum = 0; // Use long long to handle large sums
    double t1, t2;
    // Initialize the array
    for (i = 0; i < ARR_SIZE; i++) {
        a[i] = 1;
    }
    t1 = omp_get_wtime(); // Start time

    /* Do the sum in parallel */
    #pragma omp parallel default(shared) private(i, tid)
    {
        tid = omp_get_thread_num();
        numt = omp_get_num_threads();
        #pragma omp for reduction(+:sum)
        for (i = 0; i < ARR_SIZE; i++) {
            sum += a[i];
        }
    }
    t2 = omp_get_wtime(); // End time
    printf("Sum of the array elements = %lld\n", sum);
    printf("Time taken for parallel computation = %f seconds\n", t2 - t1);
    return 0;
}
```

Fundamental issues fixed in this code :

- ✓ Race condition
- ✓ Parallel for
- ✓ Cache coherency



```
D:\AMU\CLOUD-COMPUTING >
Sum of the array elements = 100000000
Time taken for parallel computation = 0.057000 seconds

-----
Process exited after 0.8635 seconds with return value 0
Press any key to continue . . . |
```

Maximizing Parallel Computing Efficiency:

If all threads in a parallel region are doing the exact same work, we aren't effectively using the power of parallel computing.

The true advantage of parallel programming comes from **dividing the work among multiple threads** so that different threads can execute different parts of a task simultaneously, leading to faster execution.

let's consider a simple example: **computing the sum of all elements in an array.**

we can divide the array into chunks, and each thread can compute the sum of a different chunk. Then, we'll combine these partial sums to get the final result.

Work Sharing

Suppose: $N=100$ and $\text{num}=4 \rightarrow N/\text{num}=25$

```
f = id*(N/num)+1  
l = (id+1)*(N/num)  
DO n=f,l,1  
  A(n) = A(n) + B  
ENDDO
```

<u>Thread 0</u>	<u>Thread 1</u>	<u>Thread 2</u>	<u>Thread 3</u>
$f=0*25+1 = 1$ $l=1*25 = 25$	$f=1*25+1=26$ $l=2*25 = 50$	$f=2*25+1=51$ $l=3*25 = 75$	$f=3*25+1=76$ $l=4*25 = 100$

Thread 0 computes elements from index 1 to 25, Thread 1 computes from index 26 to 50, etc.

The above implementation makes one big assumption. What is it?

Work Sharing

```
int chunk_size = N / num;  
int remainder = N % num;
```

```
int f, l;  
if (id < remainder) {  
    f = id * (chunk_size + 1) + 1;  
    l = f + chunk_size;  
} else {  
    f = id * chunk_size + remainder + 1;  
    l = f + chunk_size - 1;  
}
```

Work Sharing

Suppose:

- $N = 102$ (total number of elements to process)
- $num = 4$ (number of threads)

Calculations for Each Thread

Using the modified logic to calculate the range of elements (f and l) each thread will compute:

Thread 0 (id = 0):

$$f = 0 * (25 + 1) + 1 = 1$$

$$l = 1 + 26 - 1 = 26$$

Computes elements from index 1 to 26

Thread 1 (id = 1):

$$f = 1 * (25 + 1) + 1 = 27$$

$$l = 27 + 26 - 1 = 52$$

Computes elements from index 27 to 52

Thread 2 (id = 2):

$$f = 2 * 25 + 2 + 1 = 53$$

$$l = 53 + 25 - 1 = 77$$

Computes elements from index 53 to 77

Thread 3 (id = 3):

$$f = 3 * 25 + 2 + 1 = 78$$

$$l = 78 + 25 - 1 = 102$$

Computes elements from index 78 to 102

Work Sharing with OpenMP in C

Suppose we want to add a constant value B to each element of an array A of size N, and we want to distribute this task across multiple threads. If N is not divisible by the number of threads (num), OpenMP handles this automatically by distributing the iterations as evenly as possible.

```
#include <stdio.h>
#include <omp.h> // Include the OpenMP header
int main() {
    int N = 102; // Total number of elements
    int num_threads = 4; // Number of threads
    int A[N]; // Array to be modified
    int B = 5; // Constant to add to each element
    int i;
    // Initialize the array A with values from 0 to N-1
    for (i = 0; i < N; i++) {
        A[i] = i;
    }
    // Set the number of threads
    omp_set_num_threads(num_threads);
    // Parallel region with work-sharing using `#pragma omp parallel for`
    #pragma omp parallel for
    for (i = 0; i < N; i++) {
        A[i] += B; // Each thread adds B to its assigned elements
    }
    // Print the modified array to verify the result
    printf("Modified array A:\n");
    for (i = 0; i < N; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
    return 0;
}
```

Explanation of the Code

Include the OpenMP Header: `#include <omp.h>` is required to use OpenMP functions and directives.

Initialization:

We define `N = 102` (the size of the array A) and `num_threads = 4` (the number of threads to use).

The array A is initialized with values from 0 to N-1.

Setting the Number of Threads:

`omp_set_num_threads(num_threads);` sets the number of threads to 4. OpenMP will divide the work among these threads.

Parallel Region with Work-Sharing:

`#pragma omp parallel for` tells the compiler to parallelize the for loop. OpenMP automatically divides the iterations of the loop among the available threads.

Each thread will handle a portion of the iterations. If N is not divisible by `num_threads`, OpenMP will ensure that the iterations are distributed as evenly as possible.

Computing the New Values:

Inside the loop, each element `A[i]` is incremented by B.

Printing the Result:

The modified array A is printed to verify that the addition was performed correctly.

OpenMP takes care of dividing the iterations and balancing the load, making parallel programming simpler and more accessible.

Efficient Data Aggregation in OpenMP: Leveraging Reduction for Safe and Fast Computations

In OpenMP, reduction is a technique used to perform a reduction operation (like summation, multiplication, finding maximum or minimum, etc.) across all threads in a parallel region. It allows multiple threads to safely accumulate a result without race conditions.

Reduction is particularly useful in scenarios where each thread performs computations that need to be combined into a single result at the end. OpenMP provides the reduction clause to handle this.

How Reduction Works Internally

When you use the `reduction` clause:

- OpenMP creates a private copy of the reduction variable (`total_sum`) for each thread.
- Each thread performs the reduction operation (`+` in this case) on its private copy.
- At the end of the parallel region, OpenMP combines all the private copies into a single final value using the specified reduction operation.

Supported Reduction Operators in OpenMP

OpenMP supports various reduction operations:

- Addition (`+`): `reduction(+:result)`
- Multiplication (`*`): `reduction(*:result)`
- Logical AND (`&&`): `reduction(&&:result)`
- Logical OR (`||`): `reduction(||:result)`
- Bitwise AND (`&`): `reduction(&:result)`
- Bitwise OR (`|`): `reduction(|:result)`
- Bitwise XOR (`^`): `reduction(^:result)`
- Maximum (`max`): `reduction(max:result)`
- Minimum (`min`): `reduction(min:result)`

```
#include <stdio.h>
#include <omp.h> // Include OpenMP header

int main() {
    int n = 1000;
    int array[n];
    int sum = 0;

    // Initialize the array
    for (int i = 0; i < n; i++) {
        array[i] = i + 1; // array contains numbers from 1 to n
    }

    // Compute the sum in parallel
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < n; i++) {
        sum += array[i]; // Each thread will compute part of the sum
    }

    printf("Sum: %d\n", sum);
    return 0;
}
```

Explanation of the Code:

Include the OpenMP Header: The `#include <omp.h>` directive includes the OpenMP header file that provides the functions and definitions needed for parallel programming.

Parallel Directive

`#pragma omp parallel for:`

- This directive tells the compiler to parallelize the for loop. Each thread will work on a different part of the loop (different indices of the array).
- The `reduction(+:sum)` clause specifies that each thread should maintain a private copy of the sum variable. At the end of the parallel region, these private sums will be combined (reduced) into the global sum variable using the `+` operator.

Shared and Private Data:

- By default, variables declared outside the parallel region (like `array` and `n`) are shared among all threads.
- The sum variable is declared with `reduction(+:sum)`, meaning each thread will have its own private copy, and they will be summed together at the end.

Benefits

Work Sharing: Instead of every thread computing the sum of the entire array, the work is divided among all available threads, making the computation faster.

Reduction: The reduction clause handles combining the results from each thread efficiently, ensuring there are no race conditions when updating the sum variable.

Managing Data Races in OpenMP: Manual Synchronization

When performing parallel computations without using the reduction clause in OpenMP, we need to manually handle data races to ensure that the final result is correct.

This typically involves using **synchronization techniques**, such as **critical sections** or **atomic operations**, to prevent threads from updating shared data simultaneously.

In this example, we'll calculate the sum of an array without using the reduction clause and avoid data races by using a critical section to ensure that only one thread updates the shared variable at a time.

```
// Critical section to prevent data races
```

```
#include <stdio.h>
```

```
#include <omp.h> // Include the OpenMP header
```

```
int main() {
```

```
    int N = 100; // Size of the array
```

```
    int A[N]; // Array to be summed
```

```
    int i;
```

```
    int total_sum = 0; // Shared variable to hold the final sum
```

```
    // Initialize the array A with values from 1 to N
```

```
    for (i = 0; i < N; i++) {
```

```
        A[i] = i + 1;
```

```
    }
```

```
    // Parallel region without reduction, using `#pragma omp critical`
```

```
    #pragma omp parallel for
```

```
    for (i = 0; i < N; i++) {
```

```
        #pragma omp critical // Critical section to prevent data races
```

```
        {
```

```
            total_sum += A[i]; // Only one thread can execute this at a time
```

```
        }
```

```
    }
```

```
    // Print the total sum
```

```
    printf("The total sum is: %d\n", total_sum);
```

```
    return 0;
```

```
}
```

Downsides of Using critical section:

While using critical sections prevents data races, it has some drawbacks:

Performance Overhead: Entering and exiting a critical section involves overhead, which can significantly reduce parallel performance, especially if many threads frequently enter the critical section.

Potential Bottlenecks: If most of the computation is within the critical section, the parallelization benefit diminishes because threads will spend time waiting for each other to finish.

Optimizing Shared Memory Updates in OpenMP: The Efficiency of the Atomic Directive

The atomic directive in OpenMP is used to ensure that a specific memory update operation is performed atomically.

This means that the operation on a shared variable is executed without interruption, preventing race conditions.

The atomic directive is more efficient than a critical section when the operation involves a single, simple update, such as incrementing a value.

```
// Atomic operation to prevent data races
```

```
#include <stdio.h>
```

```
#include <omp.h> // Include the OpenMP header
```

```
int main() {
```

```
    int N = 100; // Size of the array
```

```
    int A[N]; // Array to be summed
```

```
    int i;
```

```
    int total_sum = 0; // Shared variable to hold the final sum
```

```
    // Initialize the array A with values from 1 to N
```

```
    for (i = 0; i < N; i++) {
```

```
        A[i] = i + 1;
```

```
    }
```

```
    // Parallel region without reduction, using `#pragma omp atomic`
```

```
    #pragma omp parallel for
```

```
    for (i = 0; i < N; i++) {
```

```
        #pragma omp atomic // Atomic operation to prevent data races
```

```
        total_sum += A[i]; // Ensures that the addition is done atomically
```

```
    }
```

```
    // Print the total sum
```

```
    printf("The total sum is: %d\n", total_sum);
```

```
    return 0;
```

```
}
```

Explanation of the Code (previous slide)

Parallel Region:

`#pragma omp parallel` for creates a parallel region where the loop iterations are distributed among multiple threads.

Using atomic Directive:

- `#pragma omp atomic` ensures that the subsequent statement, `total_sum += A[i];`, is executed atomically.
- The atomic directive only affects the specific memory update (in this case, `+=`), ensuring that only one thread at a time can perform the addition to `total_sum`.

Computing the Sum:

Each thread independently computes its partial sum from the array `A` and adds it to the shared variable `total_sum` atomically.

When to Use atomic

- Use atomic when you need to perform simple arithmetic or logical updates (e.g., `+=`, `-=`, `*=`, `/=`, `|=`, `&=`, etc.) on shared variables.
- It is particularly useful for simple updates to counters, accumulators, and flags.
- It provides better performance compared to critical for single-operation updates since atomic incurs less overhead.

Understanding OpenMP Scoping Clauses: Controlling Variable Visibility and Lifetime in Parallel Regions

private Clause

Purpose: The private clause ensures that each thread has its own independent copy of a variable. Changes made to a private variable by one thread do not affect the variable's value in other threads.

Usage: It's typically used when a variable is needed to be thread-local, such as loop indices or intermediate results.

Example:

```
#pragma omp parallel private(i)
{
    for (i = 0; i < 10; i++) {
        // Each thread has its own copy of 'i'
    }
}
```

Understanding OpenMP Scoping Clauses: Controlling Variable Visibility and Lifetime in Parallel Regions

shared Clause

Purpose: The shared clause specifies that a variable is shared among all threads within a parallel region. All threads see the same value and can read from and write to the variable.

Usage: It's used for variables that need to be accessed by all threads, such as counters or data structures that threads operate on collectively.

Example:

```
#pragma omp parallel shared(sum)
{
    // All threads have access to the same 'sum'
}
```

Understanding OpenMP Scoping Clauses: Controlling Variable Visibility and Lifetime in Parallel Regions

firstprivate Clause

Purpose: The firstprivate clause initializes each thread's private copy of a variable with the value of the variable from before the parallel region starts.

Usage: This is useful when you want each thread to start with the same initial value of a variable, but then modify it independently.

Example:

```
int x = 5;
#pragma omp parallel firstprivate(x)
{
    // Each thread has its own copy of 'x' initialized to 5
}
```

Difference Between private, firstprivate, and lastprivate

private: Each thread gets its own copy of the variable; modifications inside the parallel region are not visible outside.

firstprivate: Like private, but the copy of each variable is initialized with the value from before the parallel region.

lastprivate: Like private, but the value from the last iteration of the loop (or last section of the parallel region) is copied back to the original variable after the parallel region ends.

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {
```

```
    int x = 10; // Initial value of x
```

```
    printf("Initial value of x before parallel region: %d\n", x);
```

```
#pragma omp parallel private(x)    // Using 'private' clause
```

```
{
```

```
    int tid = omp_get_thread_num(); // Get thread ID
```

```
    x = tid; // Assign thread ID to x
```

```
    printf("Thread %d, private x = %d\n", tid, x);
```

```
}
```

```
printf("Value of x after 'private' parallel region: %d\n", x);
```

```
    x = 60; // Reset x to initial value
```

```
printf("\nInitial value of x before parallel region: %d\n", x);
```

```
#pragma omp parallel firstprivate(x)    // Using 'firstprivate' clause
```

```
{
```

```
    int tid = omp_get_thread_num(); // Get thread ID
```

```
    printf("Thread %d, firstprivate x (initially) = %d\n", tid, x);
```

```
    x += tid; // Modify x
```

```
    printf("Thread %d, firstprivate x (after modification) = %d\n", tid, x);
```

```
}
```

```
printf("Value of x after 'firstprivate' parallel region: %d\n", x);
```

```
return 0;
```

```
}
```

// Using 'private' clause and 'firstprivate' clause

// Using 'lastprivate' clause

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {
```

```
    int i;
```

```
    int x = 10; // Initial value of x
```

```
    printf("Initial value of x before parallel region: %d\n", x);
```

```
    // Using 'lastprivate' clause
```

```
    #pragma omp parallel for lastprivate(x)
```

```
    for (i = 0; i < 4; i++) {
```

```
        x = i; // Each thread modifies x
```

```
        printf("Thread %d, iteration %d, x = %d\n", omp_get_thread_num(), i, x);
```

```
    }
```

```
    // Value of x after the parallel region
```

```
    printf("Value of x after 'lastprivate' parallel region: %d\n", x);
```

```
    return 0;
```

```
}
```

// Using 'firstprivate' and 'private' clause

```
#include <stdio.h>
#include <string.h>
#include <omp.h>

int main() {
    char a[20] = "x"; // Character array with initial value "x"
    char b[20] = "y"; // Character array with initial value "y"
    int c = 3, d = 7;

    #pragma omp parallel firstprivate(a, c) private(d) shared(b) num_threads(2)
    {
        strcat(a, "k"); // Concatenate "k" to a
        c += 7; // Modify c
        printf("A becomes (%s), b is (%s) c is %d , d is %d \n", a, b, c, d);
    }
    printf("A becomes (%s), b is (%s) c is %d , d is %d \n", a, b, c, d);

    return 0;
}
```

```
D:\AMU\CLOUD-COMPUTING × + ▾
A becomes (xk), b is (y) c is 10 , d is 8
A becomes (xk), b is (y) c is 10 , d is 16218008
A becomes (x), b is (y) c is 3 , d is 7

-----
Process exited after 0.04626 seconds with return value 0
Press any key to continue . . . |
```

Understanding OpenMP Scoping Clauses: Controlling Variable Visibility and Lifetime in Parallel Regions

lastprivate Clause

Purpose: The lastprivate clause ensures that the value of the variable from the last iteration of a parallel region is copied back to the original variable after the parallel region ends.

Usage: This is useful for preserving the result of a computation that was done in the last iteration of a loop.

Example:

```
int last_value;  
#pragma omp parallel for lastprivate(last_value)  
for (int i = 0; i < 10; i++) {  
    last_value = i;  
}  
// 'last_value' will be equal to 9 after the parallel region
```

Understanding OpenMP Scoping Clauses: Controlling Variable Visibility and Lifetime in Parallel Regions

default Clause

Purpose: The default clause sets the default scoping rule for variables in a parallel region. It can be shared, private, or none.

Usage: It simplifies the specification of variable scopes if most variables in a parallel region share the same scope.

Example:

```
#pragma omp parallel default(shared)
{
    // All variables not explicitly scoped are shared
}
```

Leveraging `#pragma omp parallel sections` in OpenMP for Concurrent Task Execution

The `#pragma omp parallel sections` construct in OpenMP is used to divide work among multiple threads by defining separate code sections. Each section is executed by a different thread in parallel. This construct is particularly useful when you have multiple independent tasks that can be executed concurrently.

`#pragma omp parallel sections`

```
{  
    #pragma omp section  
    {  
        // Code block for Section 1  
    }  
    #pragma omp section  
    {  
        // Code block for Section 2  
    }  
    // Additional sections can be added as needed  
}
```

OpenMP Sections

```
#pragma omp parallel  
{
```

```
...
```

multiple threads of control □
each section assigned to a
different thread

```
#pragma omp sections
```

```
{
```

```
#pragma omp section
```

```
    X_calculation();
```

```
#pragma omp section
```

```
    y_calculation();
```

```
#pragma omp section
```

```
    z_calculation();
```

```
}
```

```
...
```

by default:
extra threads are idled

```
}
```

```
#include<stdio.h>
#include<omp.h>
int main()
{
    #pragma omp parallel sections num_threads(3)
    {
        #pragma omp section
        {
            int tid;
            tid=omp_get_thread_num();
            printf("X printed by thread with id=%d\n",tid);
        }
        #pragma omp section
        {
            int tid;
            tid=omp_get_thread_num();
            printf("Y printed by thread with id=%d\n",tid);
        }
        #pragma omp section
        {
            int tid;
            tid=omp_get_thread_num();
            printf("Z printed by thread with id=%d\n",tid);
        }
    }
    return 0;
}
```

Example: Matrix Addition and Multiplication

Consider a scenario where we have two independent tasks:

- Adding two matrices.
- Multiplying two matrices.

Both tasks can be performed in parallel since they do not depend on each other.

We'll use `#pragma omp parallel` sections to divide these tasks among threads.

Parallel Sections Construct:

`#pragma omp parallel` sections: This directive creates a parallel region and divides the work among threads.

Section 1: Adds matrices A and B and prints the result.

Section 2: Multiplies matrices A and B and prints the result.

Execution:

The `#pragma omp section` directives define blocks of code that should be executed by different threads in parallel.

Each section is executed by a different thread, leading to concurrent execution of both matrix addition and multiplication.

//Example: Matrix Addition and Multiplication

```
#include <omp.h>
#include <stdio.h>
#define SIZE 3 // Define the size of the matrix
void addMatrices(int A[SIZE][SIZE], int B[SIZE][SIZE], int C[SIZE][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }
}

void multiplyMatrices(int A[SIZE][SIZE], int B[SIZE][SIZE], int C[SIZE][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            C[i][j] = 0; // Initialize result matrix element
            for (int k = 0; k < SIZE; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

//Example: Matrix Addition and Multiplication

```
int main() {
    int A[SIZE][SIZE] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int B[SIZE][SIZE] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
    int C[SIZE][SIZE]; // Matrix for addition result
    int D[SIZE][SIZE]; // Matrix for multiplication result
```

```
// Parallel sections construct
#pragma omp parallel sections
{
```

```
    // Section 1: Matrix Addition
```

```
    #pragma omp section
```

```
    {
        addMatrices(A, B, C);
        printf("Matrix Addition Result:\n");
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                printf("%d ", C[i][j]);
            }
            printf("\n");
        }
    }
}
```

```
// Section 2: Matrix Multiplication
```

```
    #pragma omp section
```

```
    {
        multiplyMatrices(A, B, D);
        printf("\nMatrix Multiplication Result:\n");
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                printf("%d ", D[i][j]);
            }
            printf("\n");
        }
    }
```

```
} // End of parallel sections
```

```
return 0;
```

```
}
```

Key Points:

- ***Independent Tasks:*** The tasks in the sections are independent of each other, making them ideal for parallel execution.
- **Thread Synchronization:** When all sections complete, the threads synchronize and exit the parallel region.
- **Flexible Work Distribution:** You can add as many **#pragma omp section** blocks as needed within the **#pragma omp parallel sections** construct.

Ensuring Exclusive Execution with `#pragma omp single` in OpenMP: Controlling Task Execution Across Threads

The `#pragma omp single` construct in OpenMP is used within a parallel region to ensure that a block of code is executed by only one thread, while the rest of the threads in the team skip that block and continue executing subsequent code.

This is useful when you want a single thread to perform a certain task (**like initializing data or printing**) that should not be repeated by all threads.

```
#pragma omp parallel
{
    // Parallel code block

    #pragma omp single
    {
        // Code block to be executed by only one thread
    }

    // More parallel code
}
```

Example: Initialize Data in a Parallel Region

Consider a scenario where you have multiple threads performing computations in parallel, but you want only one thread to initialize a shared array before the computations begin. The `#pragma omp single` construct ensures that only one thread performs the initialization.

```
#include <omp.h>
#include <stdio.h>
#define SIZE 10
int main() {
    int arr[SIZE];
    // Parallel region
    #pragma omp parallel
    {
        // Single construct: only one thread initializes the array
        #pragma omp single
        {
            printf("Initializing array by thread %d\n", omp_get_thread_num());
            for (int i = 0; i < SIZE; i++) {
                arr[i] = i * i; // Initialize array with square of indices
            }
        }
        // All threads continue execution from here
        #pragma omp for
        for (int i = 0; i < SIZE; i++) {
            printf("Thread %d: arr[%d] = %d\n", omp_get_thread_num(), i, arr[i]);
        }
    } // End of parallel region
    return 0;
}
```

Single Construct:

#pragma omp single: This directive ensures that the code block following it is executed by only one thread in the team.

In the example, one thread (**whichever reaches this directive first**) will execute the array initialization code. The `omp_get_thread_num()` function is used to identify which thread performs the initialization.

Work-Sharing Construct:

#pragma omp for: This directive distributes the iterations of the loop among all threads in the team. After the single thread initializes the array, all threads continue and execute the loop in parallel, printing the array elements.

Output:

Only one thread will print "Initializing array by thread X," indicating that it executed the single construct.

All threads participate in printing the contents of the array, showing that the `#pragma omp single` construct is effective in ensuring that the initialization is performed only once.

// The master construct

```
#pragma omp parallel
{
    Work1();
    // This...
    #pragma omp master
    {
        Work2();
    }
    // ...is practically identical to this:
    if(omp_get_thread_num() == 0)
    {
        Work2();
    }
    Work3();
}
```

Key Points

Single Thread Execution: Only one thread (not necessarily the master thread) will execute the code within the `#pragma omp single` block.

The master construct is similar, except that the statement/block is run by the master thread, and there is no implied barrier; other threads skip the construct without waiting.

Synchronization: By default, there is an implicit barrier at the end of the `#pragma omp single` construct, meaning all threads wait until the single thread completes the execution of the block.

This can be avoided by using the `nowait` clause (e.g., `#pragma omp single nowait`).

Useful for Initialization and I/O: The `#pragma omp single` construct is often used for tasks like initializing shared data or performing input/output operations that should not be duplicated by multiple threads.

Execution synchronization

Barrier directive causes threads encountering the barrier to wait until all the other threads in the same team have encountered the barrier.

#pragma omp parallel

```
{  
    /* All threads execute this. */  
    SomeCode();  
    #pragma omp barrier  
    /* All threads execute this, but not before  
     * all threads have finished executing SomeCode().  
     */  
    SomeMoreCode();  
}
```

Note: There is an implicit barrier at the end of each parallel block, and at the end of each sections, for and single statement, unless the `nowait` directive is used.

Effects of the barrier Directive

- **Synchronization:** It synchronizes all threads in a parallel region. When a thread reaches a barrier, it waits until all other threads in the team reach that same point.
- **Data Consistency:** It ensures that any updates made by threads before the barrier are visible to all threads after the barrier. This is useful when threads need to read data that other threads have modified.
- **Prevents Race Conditions:** By ensuring all threads have completed a particular task or set of tasks before moving forward, barrier can prevent certain types of race conditions.

//Here is a simple example in C demonstrating the use of the barrier directive:

```
#include <stdio.h>
#include <omp.h>
```

```
int main() {
    int data = 0;

    #pragma omp parallel num_threads(4)
    {
        int tid = omp_get_thread_num(); // Get thread ID

        if (tid == 0) {
            data = 100; // Only thread 0 modifies the data
            printf("Thread %d sets data to %d\n", tid, data);
        }
    }
}
```

// Barrier ensures all threads wait here until thread 0 has modified 'data'

```
#pragma omp barrier
```

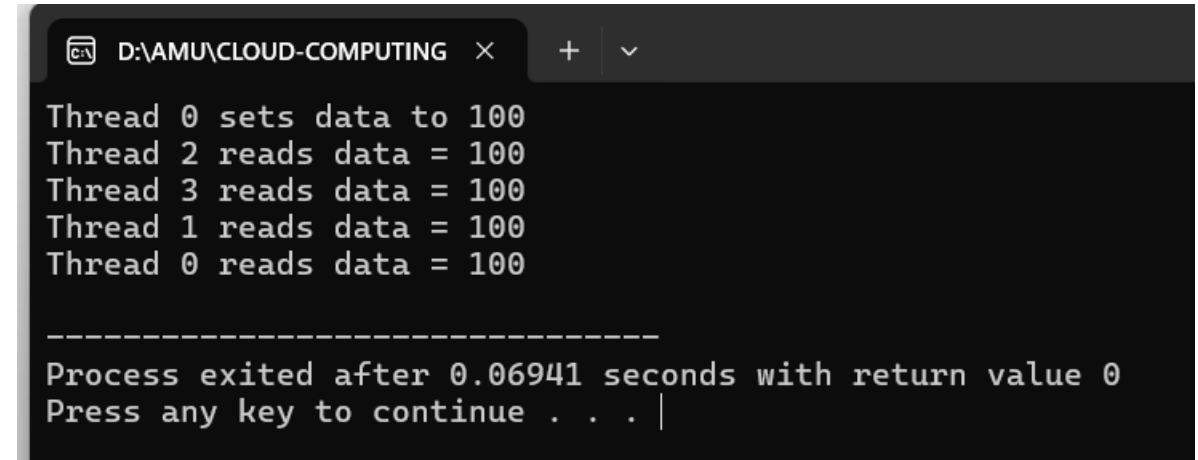
// All threads print the updated value of 'data' after the barrier

```
printf("Thread %d reads data = %d\n", tid, data);
```

```
}
```

```
return 0;
```

```
}
```



```
D:\AMU\CLOUD-COMPUTING x + v
Thread 0 sets data to 100
Thread 2 reads data = 100
Thread 3 reads data = 100
Thread 1 reads data = 100
Thread 0 reads data = 100

-----
Process exited after 0.06941 seconds with return value 0
Press any key to continue . . . |
```

Explanation

- **Parallel Region:** A parallel region is created with 4 threads using `#pragma omp parallel num_threads(4)`.

- **Modification by a Single Thread:**

Thread 0 modifies the variable `data` by setting it to 100.

- **Barrier Synchronization:**

`#pragma omp barrier` ensures all threads wait at this point until all threads in the team reach it.

This is crucial because we want to ensure that `data` is modified by thread 0 before any other thread reads it.

- **Post-Barrier Execution:**

After the barrier, all threads proceed, and each prints the value of `data`. The output shows that `data` is consistently read as 100 by all threads, demonstrating data consistency after the barrier.

Important Notes:

- **Performance Considerations:** Barriers can introduce performance overhead, especially if they are used frequently or in regions where threads have imbalanced workloads. Threads that finish early will be idle until the others catch up.
- **Implicit Barriers:** Some OpenMP directives, like `#pragma omp for` and `#pragma omp sections`, have implicit barriers at the end, which can be disabled using the `nowait` clause.
- **Deadlock Risk:** Improper use of barrier can cause deadlocks if some threads never reach the barrier point, so it's essential to use it carefully.

The nowait clause in OpenMP is used to remove the implicit barrier that normally occurs at the end of certain constructs, like `#pragma omp for`, `#pragma omp sections`, or `#pragma omp single`.

By default, these constructs have an implicit barrier at the end, meaning that all threads must wait until every thread has completed its work before proceeding to the next part of the code.

Using `nowait` allows threads to continue without waiting for others, potentially improving performance by reducing idle time.

When to Use `nowait`

- **Reduce Synchronization Overhead:** If you have multiple independent tasks that do not need to synchronize after completion, removing the barrier with `nowait` can reduce the overhead.
- **Improve Load Balancing:** In situations where work distribution among threads is imbalanced, using `nowait` can allow faster threads to move on to other tasks without waiting for slower threads to finish.

```
#include <stdio.h> // Parallel loop without 'nowait', barrier at the end
```

```
#include <omp.h>
```

```
int main() {
```

```
    int i;
```

```
    #pragma omp parallel num_threads(4)
```

```
    {
```

```
        int tid = omp_get_thread_num(); // Get the thread ID
```

```
        // Parallel loop without 'nowait', barrier at the end
```

```
        #pragma omp for
```

```
        for (i = 0; i < 8; i++) {
```

```
            printf("Thread %d processing iteration %d\n", tid, i);
```

```
        }
```

```
        // This code block will not execute immediately after the loop, waiting for other threads
```

```
        printf("Thread %d completed loop and continues immediately.\n", tid);
```

```
    }
```

```
    return 0;
```

```
}
```

```
D:\AMU\CLOUD-COMPUTING x + v
Thread 2 processing iteration 4
Thread 2 processing iteration 5
Thread 0 processing iteration 0
Thread 0 processing iteration 1
Thread 1 processing iteration 2
Thread 1 processing iteration 3
Thread 3 processing iteration 6
Thread 3 processing iteration 7
Thread 2 completed loop and continues immediately.
Thread 0 completed loop and continues immediately.
Thread 1 completed loop and continues immediately.
Thread 3 completed loop and continues immediately.

-----
Process exited after 0.019 seconds with return value 0
Press any key to continue . . . |
```

```
#include <stdio.h> // Parallel loop with 'nowait', no barrier at the end
```

```
#include <omp.h>
```

```
int main() {
```

```
    int i;
```

```
    #pragma omp parallel num_threads(4)
```

```
    {
```

```
        int tid = omp_get_thread_num(); // Get the thread ID
```

```
        // Parallel loop with 'nowait', no barrier at the end
```

```
        #pragma omp for nowait
```

```
        for (i = 0; i < 8; i++) {
```

```
            printf("Thread %d processing iteration %d\n", tid, i);
```

```
        }
```

```
        // This code block will execute immediately after the loop without waiting for other threads
```

```
        printf("Thread %d completed loop and continues immediately.\n", tid);
```

```
    }
```

```
    return 0;
```

```
}
```

```
D:\AMU\CLOUD-COMPUTING x + v
Thread 0 processing iteration 0
Thread 0 processing iteration 1
Thread 0 completed loop and continues immediately.
Thread 2 processing iteration 4
Thread 2 processing iteration 5
Thread 2 completed loop and continues immediately.
Thread 3 processing iteration 6
Thread 3 processing iteration 7
Thread 3 completed loop and continues immediately.
Thread 1 processing iteration 2
Thread 1 processing iteration 3
Thread 1 completed loop and continues immediately.

-----
Process exited after 0.04756 seconds with return value 0
Press any key to continue . . . |
```

Synchronization: Barrier

```
#pragma omp parallel private(id)
{
```

```
    id=omp_get_thread_num();
    A[id] = big_calc1(id);
    #pragma omp barrier
```

explicit barrier

```
    #pragma omp for
```

```
    for(i=0;i<N;i++)
```

```
    {
        C[i]=big_calc3(i,A);
    }
```

implicit barrier at end
of parallel region

```
    #pragma omp for nowait
```

```
    for(i=0;i<N;i++)
```

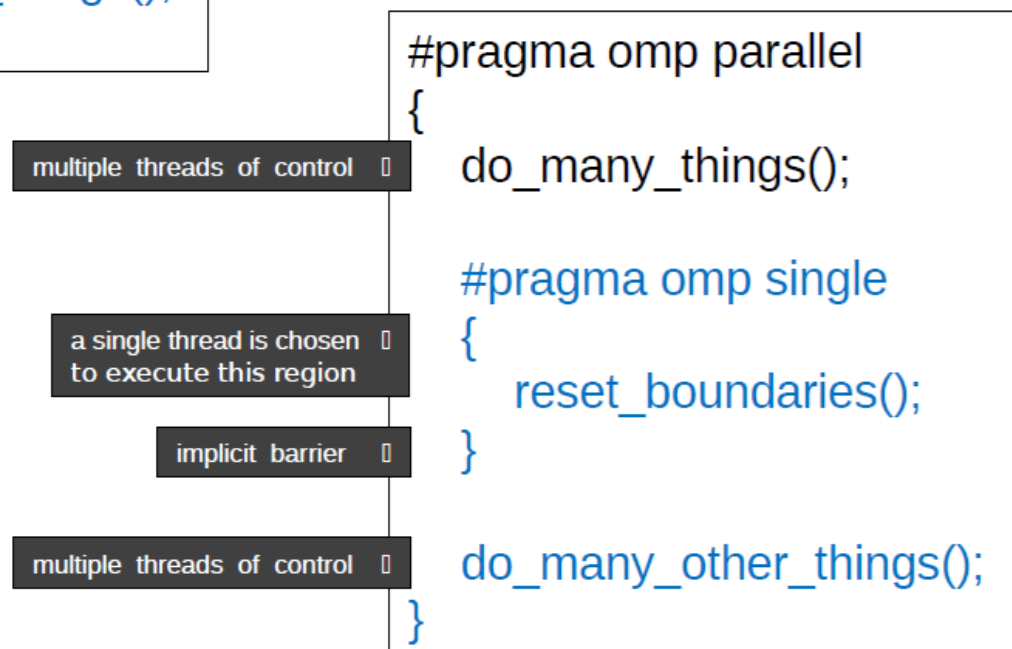
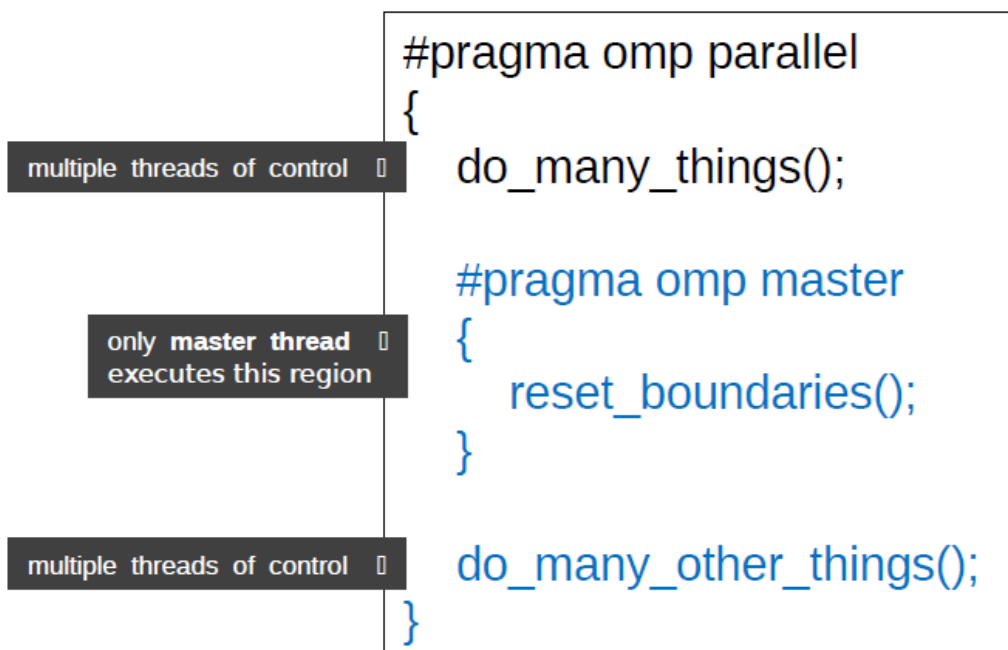
```
    {
        B[i]=big_calc2(C, i);
    }
```

no barrier!
nowait cancels barrier creation

```
    A[id] = big_calc4(id);
```

```
}
```

Synchronization: Master and Single



The ordered clause in OpenMP is used to ensure that specific blocks of code within a parallel loop are executed in the same order as they would be in a sequential execution. This is particularly useful when you have computations that must occur in a particular order within a parallel region.

Purpose of the ordered Clause

- Order Enforcement: The ordered clause ensures that certain sections of code are executed in the order of the loop iterations, even though the loop itself is executed in parallel.
- Combination with **#pragma omp for** and **#pragma omp parallel for**: The ordered clause is typically used with for loops to maintain the sequential order of operations.

```
#pragma omp parallel for ordered
for (int i = 0; i < N; i++) {
    // Parallel part of the loop

    #pragma omp ordered
    {
        // Code block that must execute in order
    }
}
```

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i;
    int N = 10;

    // Parallel loop with 'ordered' clause
    #pragma omp parallel for ordered num_threads(4)
    for (i = 0; i < N; i++) {
        // Simulate some work done in parallel
        printf("Thread %d processing iteration %d in parallel\n", omp_get_thread_num(), i);

        // Ordered section to print the results in order
        #pragma omp ordered
        {
            printf("Ordered output: iteration %d\n", i);
        }
    }

    return 0;
}
```

Explanation

Parallel for Loop with ordered Clause:

The loop is parallelized with `#pragma omp parallel for ordered`. The ordered clause is specified after `for` to indicate that an ordered block will be present in the loop.

Parallel Section:

Each thread processes different iterations of the loop concurrently, as indicated by the first `printf` statement. This part of the code runs in parallel.

Ordered Section:

The `#pragma omp ordered` directive is placed around the code that needs to be executed in a sequential order (i.e., the second `printf` statement). This section ensures that the output will appear in order according to the loop index `i`.

